Database Users, Roles, Privileges, and Access Control

Group 11 (BoardTrack: Boarding House & Rental Management System)

Section	Contents
Cover and links	**GROUP 12: Amoranto, Denielle; Eurescial, Eunice; Francisco, Helix** **Project Assigned:** BoardTrack: Boarding House & Rental Management System **Repository Link: https://github.com/karlrobertc-art/BoardTrack** **Our Group Repository:**
Projects set up	**The BoardTrack repository was used as the basis for implementing the database access-control plan. The project repository served as the reference for preparing the PostgreSQL Role-Based Access Control (RBAC) implementation.** **The first step was to examine the existing project files and understand the available database-related configurations and application structure. The repository was reviewed to identify the database management system, existing database configuration, and database objects that would require access restrictions.** **The database management system used for this activity was PostgreSQL. The default schema considered for the access-control implementation was the public schema. The existing database structure was intended to be retained, while the access-control implementation was prepared separately to avoid unnecessary modifications to the original project functionality.** **The database was inspected using PostgreSQL queries to identify the current database name, existing schemas, tables, columns, sequences, and database roles. This information served as the basis for preparing the role assignments and privilege configuration.** **After reviewing the database setup and access-control requirements, a separate folder named Activity-4 was prepared to organize the SQL scripts and documentation.** **The following files were created:** **00_inspect_database.sql**

	01_create_roles.sql **02_grant_privileges.sql** **03_revoke_privileges.sql** **04_test_privileges.sql** **access_control_plan.md** **screenshots/** **These files were organized to make the implementation easier to understand, execute, verify, and maintain.**
Access-control plan	**The access-control plan was created to determine which database operations should be allowed for each user role within the BoardTrack system.** **The implementation follows the Principle of Least Privilege, meaning that each database role should only receive the permissions necessary for its assigned responsibilities.** **Five PostgreSQL group roles were established to represent the different responsibilities within the BoardTrack system.** **Administrator Role (boardtrack_admin_role)** **The administrator role is responsible for managing and maintaining the BoardTrack database.** **Its responsibilities include viewing database records, inserting new records, updating existing information, deleting records when necessary, and managing database access permissions.** **The administrator role receives ALL PRIVILEGES on all existing tables and sequences within the public schema. It also receives CREATE permission on the public schema.** **This role is intended only for authorized database administrators and should not be used for ordinary application operations.** **Application Role (boardtrack_app_role)** **The application role is intended for the backend application responsible**

	for communicating with the BoardTrack database. **Its responsibilities include retrieving database records, inserting new information, updating existing records, and deleting records when required by the application's functionality.** **The application role receives SELECT, INSERT, UPDATE, and DELETE privileges on all existing tables within the public schema.** **It also receives USAGE and SELECT privileges on existing sequences to support database operations involving sequence-generated values.** **However, the application role is restricted from performing TRUNCATE, REFERENCES, and TRIGGER operations.** **This role is intended to support regular application database operations without providing unnecessary administrative privileges.** **Manager Role (boardtrack_manager_role)** **The manager role is intended for users responsible for overseeing and managing BoardTrack records.** **Its responsibilities include viewing existing information, inserting new records, updating existing information, and deleting records when necessary.** **The manager role receives SELECT, INSERT, UPDATE, and DELETE privileges on all existing tables within the public schema.** **It also receives USAGE and SELECT privileges on existing sequences to support applicable database operations.** **The manager role is restricted from performing TRUNCATE, REFERENCES, and TRIGGER operations.** **This allows managers to perform regular record-management operations without having full database administration privileges.** **Member Role (boardtrack_member_role)** **The member role is intended for users who need limited access to the BoardTrack database.** **Its responsibilities include viewing permitted records, inserting new information, and updating existing records when allowed by the system's functionality.**

The member role receives SELECT, INSERT, and UPDATE privileges on all existing tables within the public schema.

It also receives USAGE and SELECT privileges on existing sequences.

However, the member role is restricted from performing DELETE and TRUNCATE operations.

This helps prevent unauthorized removal of database records while still allowing members to perform permitted database operations.

Read-Only Role (boardtrack_readonly_role)

The read-only role is intended for authorized users who only need to view database information.

Its responsibilities include viewing existing records, inspecting database information, and reviewing records for monitoring and reporting purposes.

The read-only role receives SELECT privileges on all existing tables within the public schema.

It cannot insert, update, delete, or truncate database records. It is also restricted from modifying or using existing database sequences.

This ensures that users assigned to this role can inspect database information without making unauthorized modifications.

Access-Control Matrix

The following matrix summarizes the intended database permissions assigned to each group role.

Database Operations

Database Operations	Administrator	Application	Manager	Member	Read-Only
SELECT	Allowed	Allowed	Allowed	Allowed	Allowed

	INSERT	**Allowed**	**Allowed**	**Allowed**	**Allowed**	**Denied**
	UPDATE	**Allowed**	**Allowed**	**Allowed**	**Allowed**	**Denied**
	DELETE	**Allowed**	**Allowed**	**Allowed**	**Denied**	**Denied**
	TRUNCATE	**Allowed**	**Denied**	**Denied**	**Denied**	**Denied**
	REFERENCES	**Allowed**	**Denied**	**Denied**	**Not granted**	**Denied**
	TRIGGER	**Allowed**	**Denied**	**Denied**	**Not granted**	**Denied**
	Sequence Privileges	**Allowed**	**Allowed**	**Allowed**	**Allowed**	**Denied**
	CREATE in public schema	**Allowed**	**Not granted**	**Not granted**	**Not granted**	**Not granted**
	The permissions listed above represent the intended access-control configuration based on the prepared SQL scripts. **The administrator receives full privileges on existing tables and sequences. The application and manager roles receive CRUD privileges, while the member role receives SELECT, INSERT, and UPDATE privileges. The read-only role is limited to SELECT operations.** **The roles should also be reviewed for inherited privileges because PostgreSQL permissions can be obtained through other role**					

memberships and grants.

Actual effective privileges must be verified through PostgreSQL testing.

Database Users and Role Assignments

The following PostgreSQL database login accounts were prepared:

Database User	Assigned Group Role
boardtrack_admin	boardtrack_admin_role
boardtrack_app	boardtrack_app_role
boardtrack_manager	boardtrack_manager_role
boardtrack_member	boardtrack_member_role
boardtrack_viewer	boardtrack_readonly_role

The five group roles were created using the NOLOGIN attribute. This means they function as permission groups and cannot be used as direct login accounts.

The login accounts were created using the LOGIN attribute and assigned to their corresponding group roles through PostgreSQL GRANT statements.

This allows database users to inherit the permissions assigned to their respective group roles.

	The administrator account is intended for database administration, the application account is intended for backend database operations, the manager account is intended for managing records, the member account is intended for limited database operations, and the viewer account is intended for read-only access. **The separation of login accounts and group roles makes permission management easier because privileges can be assigned to group roles instead of being individually configured for every login account.**
SQL scripts summary	**Five SQL scripts were prepared to inspect, implement, and verify the database access-control configuration.** **File 1: 00_inspect_database.sql** **The first SQL script was used to inspect the existing PostgreSQL database structure before implementing the access-control configuration.** **The script contains queries that display the current database name, current database user, existing tables, table columns, sequences, and PostgreSQL roles.** **The inspection focused on identifying:** ● **Current PostgreSQL database name.** ● **Existing tables in the public schema.** ● **Table column names and data types.** ● **Existing database sequences.** ● **Current PostgreSQL roles.** ● **Login and administrative attributes of existing roles.** **This step was important because the access-control implementation needed to be based on the actual database structure instead of using placeholder table names.** **The inspection also helped identify the database objects that would receive the assigned privileges and helped prevent errors caused by referencing tables that do not exist.** **File 2: 01_create_roles.sql** **The second SQL script was used to create the five PostgreSQL**

	group roles and their corresponding login accounts. **The following group roles were created:** ● **boardtrack_admin_role** ● **boardtrack_app_role** ● **boardtrack_manager_role** ● **boardtrack_member_role** ● **boardtrack_readonly_role** **The following database login accounts were also created:** ● **boardtrack_admin** ● **boardtrack_app** ● **boardtrack_manager** ● **boardtrack_member** ● **boardtrack_viewer** **Each login account was assigned to its corresponding group role using PostgreSQL GRANT statements.** **The group roles were created using the NOLOGIN attribute, while the database login accounts were created using the LOGIN attribute.** **This approach allows permissions to be managed through group roles rather than assigning privileges individually to every database account.** **The administrator account is intended for database administration, the application account is intended for application backend operations, the manager account is intended for record management, the member account is intended for limited database access, and the viewer account is intended for read-only access.** **Separate passwords should be configured for each login account to help protect database access.** **File 3: 02_grant_privileges.sql** **The third SQL script was used to assign the required database permissions to each group role.** **The implementation uses the public schema as the target schema**

	for the initial privilege configuration. **All five group roles were given USAGE permission on the public schema so they could access database objects for which they have the appropriate privileges.** **The administrator role received ALL PRIVILEGES on all existing tables in the public schema. It also received ALL PRIVILEGES on existing sequences and CREATE permission on the public schema.** **The application role received SELECT, INSERT, UPDATE, and DELETE privileges on all existing tables in the public schema.** **The manager role received SELECT, INSERT, UPDATE, and DELETE privileges on all existing tables in the public schema.** **The member role received SELECT, INSERT, and UPDATE privileges on all existing tables in the public schema.** **The read-only role received SELECT privileges on all existing tables in the public schema.** **The application, manager, and member roles also received USAGE and SELECT privileges on existing sequences.** **The use of ON ALL TABLES IN SCHEMA public allows PostgreSQL to assign permissions to all existing tables without manually specifying each table name.** **Similarly, ON ALL SEQUENCES IN SCHEMA public allows privileges to be assigned to all existing sequences in the specified schema.** **The script also includes ALTER DEFAULT PRIVILEGES statements for future tables created by the role executing the script.** **These statements help simplify privilege assignment when working with a database containing multiple tables and sequences.** **It is important to note that default privileges apply to objects created by the configured object-owning role. If another database owner creates future tables, default privileges must also be configured for that owner.**

	File 4: 03_revoke_privileges.sql **The fourth SQL script was used to remove unnecessary privileges from restricted roles.** **The member role was restricted from performing DELETE and TRUNCATE operations on existing database tables.** **The application role was restricted from performing TRUNCATE, REFERENCES, and TRIGGER operations.** **The manager role was also restricted from performing TRUNCATE, REFERENCES, and TRIGGER operations.** **The read-only role was restricted from performing INSERT, UPDATE, DELETE, TRUNCATE, REFERENCES, and TRIGGER operations.** **The read-only role was also restricted from using or updating existing database sequences.** **These restrictions help prevent ordinary users from performing unnecessary database operations or making unauthorized modifications.** **The administrator role retains its assigned table and sequence privileges, while the application, manager, member, and read-only roles are limited according to their assigned responsibilities.** **This helps implement the Principle of Least Privilege by preventing unnecessary access rights.** **It is important to note that PostgreSQL REVOKE statements remove specific grants but do not automatically cancel permissions inherited from other roles or separate grants.** **Therefore, the effective privileges of each login account must also be reviewed during testing.** **File 5: 04_test_privileges.sql** **The fifth SQL script was used to check whether the assigned database permissions were properly applied.** **The script contains queries that display the assigned database**

	roles, verify role memberships, and inspect table privileges using PostgreSQL privilege-checking functions. **The verification focused on checking whether:** ● **The administrator has SELECT privileges.** ● **The administrator has INSERT privileges.** ● **The administrator has UPDATE privileges.** ● **The administrator has DELETE privileges.** ● **The application role has CRUD privileges.** ● **The manager role has CRUD privileges.** ● **The member role can view existing records.** ● **The member role can insert records.** ● **The member role can update records.** ● **The member role cannot delete records.** ● **The read-only role can view existing records.** ● **The read-only role cannot insert records.** ● **The read-only role cannot update records.** ● **The read-only role cannot delete records.** **These checks help confirm that each role follows the intended access-control configuration.** **The verification script uses PostgreSQL functions such as has_table_privilege() to determine whether specific database roles have permission to perform certain operations.** **The privilege-checking queries evaluate effective permissions, including permissions inherited through role memberships.** **Additional testing using separate login accounts is necessary to confirm that unauthorized operations are actually blocked.**
Verification results	**After preparing the SQL scripts, they were intended to be executed in the PostgreSQL Query Tool through pgAdmin 4.** **The scripts were organized to be executed in the correct order to avoid permission-related errors.** **First Verification: Database Inspection** **The first script was executed to inspect the existing database structure.**

	The database name and existing tables were checked using PostgreSQL queries. **The following database information was identified through the inspection process:** **Item** — **Result** **Database Name** — **Actual PostgreSQL database name** **Schema Name** — **public** **Database Management System** — **PostgreSQL** **Project** — **BoardTrack** **Expected result:** **The database inspection queries should successfully display the current database name, existing tables, columns, sequences, and PostgreSQL roles.** **The actual database name and table structure should be recorded based on the query results.** **Second Verification: Role Creation** **The second script was executed to create the five group roles and their corresponding login accounts.** **A verification query was used to display the created roles from the PostgreSQL system catalog.** **Expected result:** **All five group roles and their corresponding login accounts should**

	appear in the query results. **The group roles should have the NOLOGIN attribute, while the corresponding login accounts should have the LOGIN attribute.** **The login accounts should also appear as members of their corresponding group roles.** **Third Verification: Privilege Assignment** **The third script was executed to assign the required schema, table, and sequence privileges.** **The execution messages were checked to determine whether the GRANT statements completed successfully.** **Expected result:** **Each role should receive the permissions defined in the access-control plan.** **The administrator should receive full privileges on existing tables and sequences.** **The application and manager roles should receive SELECT, INSERT, UPDATE, and DELETE privileges.** **The member role should receive SELECT, INSERT, and UPDATE privileges.** **The read-only role should receive SELECT privileges only.** **Fourth Verification: Privilege Restrictions** **The fourth script was executed to remove unnecessary privileges from restricted roles.** **Expected result:** **The application role should not have TRUNCATE privileges.** **The manager role should not have TRUNCATE privileges.** **The member role should not have DELETE privileges.** **The member role should not have TRUNCATE privileges.**

	The read-only role should not have INSERT privileges. **The read-only role should not have UPDATE privileges.** **The read-only role should not have DELETE privileges.** **The read-only role should not have TRUNCATE privileges.** **The read-only role should not have sequence privileges.** **Fifth Verification: Permission Checking** **The fifth script was executed to inspect the assigned permissions using PostgreSQL privilege-checking functions.** **The following results represent the expected permission configuration:**

Verification	Expected Result
Administrator can SELECT records	**TRUE**
Administrator can INSERT records	**TRUE**
Administrator can UPDATE records	**TRUE**
Administrator can DELETE records	**TRUE**
Application can SELECT records	**TRUE**

	Application can INSERT records	**TRUE**
	Application can UPDATE records	**TRUE**
	Application can DELETE records	**TRUE**
	Manager can SELECT records	**TRUE**
	Manager can INSERT records	**TRUE**
	Manager can UPDATE records	**TRUE**
	Manager can DELETE records	**TRUE**
	Member can SELECT records	**TRUE**
	Member can INSERT records	**TRUE**
	Member can UPDATE records	**TRUE**
	Member can DELETE records	**FALSE**
	Read-Only can SELECT records	**TRUE**

Read-Only can INSERT records	**FALSE**
Read-Only can UPDATE records	**FALSE**
Read-Only can DELETE records	**FALSE**

The expected results demonstrate that the administrator has the intended database privileges, the application and manager roles can perform CRUD operations, the member role has limited modification privileges, and the read-only role is restricted to viewing records.

Note: These are expected results based on the prepared SQL scripts, not confirmed execution results. Actual results must be recorded after running the scripts and testing the permissions using the assigned PostgreSQL accounts.

Screenshots

The following screenshots should be included as supporting evidence of the implementation and verification process:

Screenshot 1: G11_database_inspection.png

Screenshot 2: G11_create_roles.png

Screenshot 3: G11_grant_privileges.png

Screenshot 4: G11_revoke_privileges.png

Screenshot 5: G11_test_privileges.png

Screenshot 6: G11_role_memberships.png

Screenshot 7: G11_viewer_access_restrictions.png

Screenshot 8: G11_member_access_restrictions.png

	The screenshots serve as supporting evidence of the database inspection, role creation, privilege assignment, privilege restrictions, and permission verification. **Additional screenshots of the BoardTrack database structure and PostgreSQL database setup may also be included to demonstrate that the existing database was successfully identified and used for the activity.**
Issues and fixes	**During the preparation and implementation of the access-control configuration, several possible issues were considered and addressed.** **Database Name Mismatch** **One possible error encountered during the preparation of the grant privileges script was:** **ERROR: database "your_database_name" does not exist.** **This happens when a placeholder database name is used and does not match the actual PostgreSQL database name.** **To resolve this issue, the current database name can be identified using the following query:** **SELECT current_database();** **The actual database name should then be used when preparing the GRANT CONNECT statement, if database-level CONNECT permissions are included.** **Existing Database Roles** **Running role creation commands repeatedly may cause errors if the roles already exist.** **To prevent this issue, the existing roles should be checked before executing the CREATE ROLE statements again.** **If the roles already exist, their current configuration should be reviewed before making changes.** **This helps avoid duplicate role creation errors when executing the SQL scripts.**

	Incorrect Execution Order **Granting privileges before creating the required roles can result in errors.** **To prevent this issue, the scripts were organized and executed in the following order:** **00_inspect_database.sql** **01_create_roles.sql** **02_grant_privileges.sql** **03_revoke_privileges.sql** **04_test_privileges.sql** **This ensures that the existing database structure is inspected first, the required roles are created before permissions are assigned, and the final privileges are checked afterward.** **Insufficient Database Permissions** **Some PostgreSQL commands require appropriate administrative privileges.** **The scripts should be executed using an authorized PostgreSQL administrator account or a database owner with sufficient privileges to ensure that role creation and privilege assignment commands can be completed successfully.** **Incorrect Table Names** **Granting privileges to tables that do not exist can result in relation errors.** **To avoid this issue, the grant privileges script uses:** **GRANT SELECT, INSERT, UPDATE, DELETE** **ON ALL TABLES IN SCHEMA public** **TO boardtrack_app_role;** **This allows PostgreSQL to apply the specified permissions to all existing tables in the public schema without manually entering individual table names.**

	Similarly, the sequence privilege statements use: **GRANT ALL PRIVILEGES** **ON ALL SEQUENCES IN SCHEMA public** **TO boardtrack_admin_role;** **This allows permissions to be assigned to existing sequences in the specified schema.** **Database Connection and Schema Access** **Database-level CONNECT privileges do not automatically provide access to tables.** **The appropriate schema and table privileges must also be granted.** **To address this, the access-control configuration includes USAGE permission on the public schema and the required table and sequence privileges.** **If CONNECT permissions are required, they should also be granted separately on the actual database.** **Future Database Objects** **Newly created tables and sequences may not automatically receive the same permissions as existing objects.** **Default privileges should be configured for the correct object-owning role when necessary.** **The ALTER DEFAULT PRIVILEGES statements included in the grant privileges script establish default table permissions for future tables created by the configured owner.** **This helps ensure that future database objects receive the intended access permissions.** **Role Inheritance and Permission Conflicts** **PostgreSQL allows users to inherit permissions through role memberships and other grants.** **Revoking a privilege from one group role does not necessarily remove the same privilege if the user receives it through another role.** **To resolve this issue, role memberships and effective privileges should**

	be reviewed during testing. **This helps ensure that the actual permissions of each login account match the intended access-control configuration.** **Login Account Passwords** **Creating a PostgreSQL role with the LOGIN attribute does not automatically establish a secure password.** **To resolve this issue, separate passwords should be configured for each login account using PostgreSQL password-management commands.** **Sensitive database credentials should not be stored in publicly accessible repositories.** **Application and Database Permission Differences** **The application role receives CRUD privileges to support regular application operations.** **However, these permissions should be reviewed against the actual BoardTrack application functionality.** **Only the required permissions should be maintained in a production environment.** **This helps reduce unnecessary database access and improves the implementation of the Principle of Least Privilege.**
Feedback to owners	**The BoardTrack project can benefit from implementing Role-Based Access Control to improve database security and organize access permissions according to user responsibilities.** **The following recommendations are provided to the project owners:** **1. Maintain separate database roles for administrators, application operations, managers, members, and read-only users.** **2. Avoid using administrator-level database credentials for regular application activities.** **3. Assign actual login accounts to the appropriate group roles.** **4. Use strong passwords and protect database credentials.** **5. Review database permissions whenever new tables and application features are added.** **6. Restrict application access to only the database operations required by the backend.**

	7. **Limit member permissions to the specific operations necessary for their responsibilities.** 8. **Use read-only accounts for users who only need to view database information.** 9. **Apply appropriate database privileges to newly created tables and sequences.** 10. **Perform regular database backups to help prevent data loss.** 11. **Avoid granting unnecessary administrative privileges to application accounts.** 12. **Regularly test database access restrictions to ensure that unauthorized operations are prevented.** 13. **Keep database credentials and sensitive configuration files out of public GitHub repositories.** 14. **Review database roles and privileges regularly to prevent unnecessary access.** 15. **Implement appropriate application-level validation to protect system records and user information.** **These recommendations can help improve the security, reliability, and maintainability of the BoardTrack system.**
AI usage declaration	**Artificial Intelligence (AI) tools were used as a supplementary resource during the activity to assist in understanding PostgreSQL role creation, database privileges, SQL syntax, and documentation preparation.** **AI assistance was used to help organize the SQL scripts, explain the purpose of each access-control role, prepare the database access-control plan, and identify possible implementation issues and their corresponding solutions.** **The generated SQL commands and documentation were prepared based on the available BoardTrack project information and the access-control requirements of the activity.** **The SQL scripts must be validated through PostgreSQL execution using pgAdmin 4. The actual database name, tables, role memberships, and assigned privileges must be checked before final submission.** **The student remains responsible for checking the correctness of the implementation, understanding the assigned permissions, and verifying the actual execution results.** **All passwords and other sensitive credentials must be excluded from the submitted repository, screenshots, and documentation.**